Module 8: Portfolio Project Part II

Sunita Gurung

Colorado State University Global

CSC450: Programming III

Dr. Jack Li

June 07, 2026

Portfolio Project Part II - Concurrency Concepts

Screenshots of the application executing the application

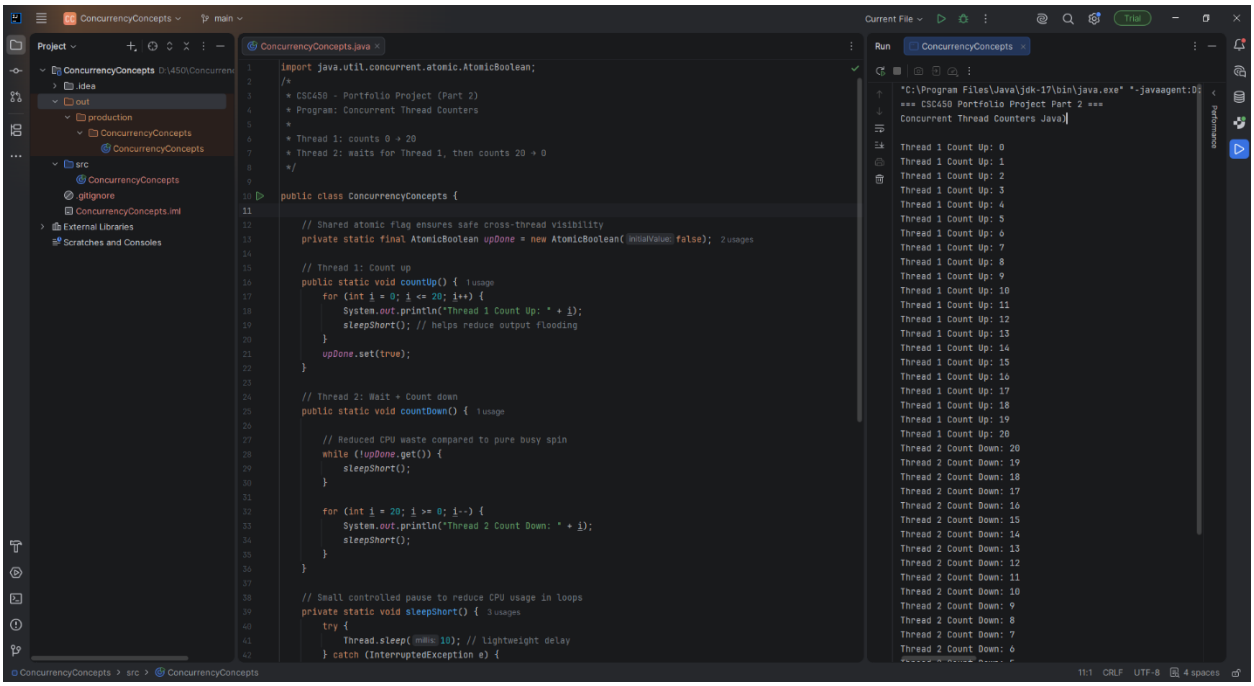

Figure 1: ConcurrencyConcepts.java1

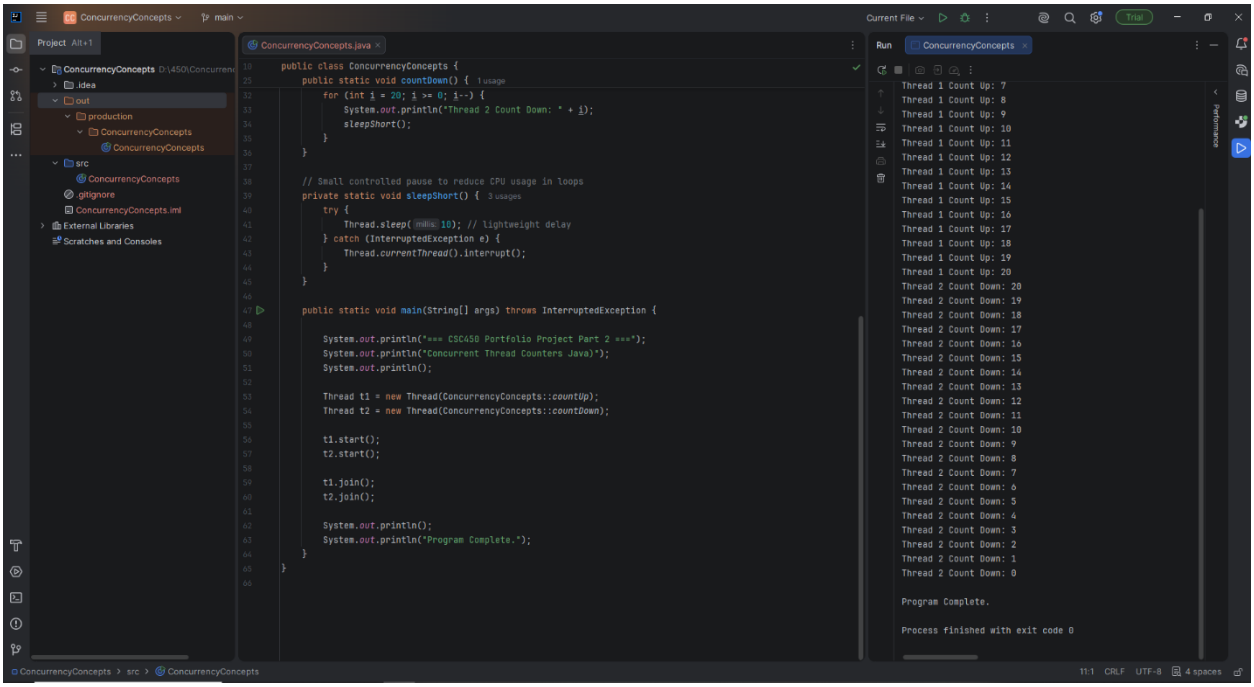

Figure 2: ConcurrencyConcepts.java2

Program analysis

Performance Issues with Concurrency

Concurrency can improve responsiveness in Java programs, but it also introduces several performance challenges that become noticeable even in a simple two-thread example. One major concern is the possibility of race conditions, where multiple threads access shared data at the same time and produce inconsistent results. This program avoids that issue by using an Atomic Boolean, which ensures that updates to the shared flag are both atomic and visible across threads. Another performance issue is busy waiting, without short sleep inside the loop, the second thread would continuously check the flag and waste CPU cycles. Adding a brief sleep reduces unnecessary spinning, though it introduces a very small delay before the thread notices the flag change. Console output also becomes a bottleneck because “System.out.println” is synchronized and significantly slower than the counting operations themselves, limiting any real parallel speedup. Even thread creation has overheard, and while it is negligible in a two-thread program, larger systems often rely on thread pools to avoid repeatedly creating and destroying thread.

Vulnerabilities exhibited with use of strings

Although the program only uses strings for printing output, Java’s string behavior still raises important security considerations. Java protects against classic buffer overflows because string operations are bounds-checked, preventing memory corruption that is common in languages like C and C++. Strings are also immutable, which makes them safe for concurrent reads because they cannot be modified after creation. However, immutability becomes a drawback when handling sensitive data such as passwords, since the contents of a String cannot be cleared from memory immediately; instead, developers are encouraged to use mutable character arrays so the data can be overwritten. Strings can also introduce “injection vulnerabilities” if user input is inserted directly into SQL queries, system

commands, or log files. Java does not automatically prevent this, so secure coding practices like input validation and using PreparedStatement are essential in real application.

Security of the data types exhibited

The data types used in this program contribute significantly to its overall safety. The shared flag is stored in an AtomicBoolean, which ensures atomic updates and proper visibility between threads, preventing synchronization errors that could occur with a regular Boolean. The counters use the int type, which is sage in this context because the values remain well within the valid range avoiding overflow. Still, it is important to understand the values remain well within the valid range, avoiding overflow, unlike C++, where signed overflow is undefined behavior. When overflow detection is required, Java provides methods like “Math.addExact()” that throw exceptions instead of wrapping. Beyond individual types, Java’s strong type system and JVM verification process add another layer of protection by preventing unsafe casts, memory corruption, and type confusion attacks that are possible in lower-level languages. These features collectively make Java’s type system more secure and predictable for concurrent programming.

Source Code

Java Source (ConcurrencyConcepts.java)

```
import java.util.concurrent.atomic.AtomicBoolean;

public class ConcurrencyConcepts {

   private static final AtomicBoolean upDone = new AtomicBoolean(false);

   public static void countUp() {
      for (int i = 0; i <= 20; i++) {
```

```
            System.out.println("Thread 1 Count Up: " + i);
            sleepShort();
        }
        upDone.set(true);
    }

    public static void countDown() {
        while (!upDone.get()) {
            sleepShort();
        }
        for (int i = 20; i >= 0; i--) {
            System.out.println("Thread 2 Count Down: " + i);
            sleepShort();
        }
    }

    private static void sleepShort() {
        try {
            Thread.sleep(10);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    public static void main(String[] args) throws InterruptedException {
        System.out.println("=== CSC450 Portfolio Project Part 2 ===");
        System.out.println("Concurrent Thread Counters (Improved Java Version)");
```

```
        System.out.println();

        Thread t1 = new Thread(ConcurrencyConcepts::countUp);
        Thread t2 = new Thread(ConcurrencyConcepts::countDown);

        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println();
        System.out.println("Program Complete.");
    }
}
```

Comparison between Java and C++

The Java and C++ versions of my concurrency program perform the same task, but the two languages differ significantly in how they handle threading, memory, performance, and overall program safety. Although both implementations create two threads, coordinate them with an atomic flag, and ensure that one thread completes before the other begins, the underlying mechanisms that support these operations behave very differently in each language. These differences influence not only how efficiently the program runs but also how vulnerable it is to security threats. Understanding these distinctions is essential for evaluating the strengths and weaknesses of each language in the context of concurrent programming.

In the C++ implementation, the program uses “std::thread,” which maps directly to the operating system’s native threading model. This gives C++ very low overhead because the compiled program runs as native machine code without any intermediate runtime layer. The operating system schedules the threads directly, and the program interacts with hardware resources in a straightforward manner. This design allows C++ to achieve high performance, especially in applications where low-level control and minimal latency are important. The Java version, by contrast, uses the Thread class managed by the Java Virtual Machine. The JVM introduces additional layers of abstraction, including bytecode interpretation, just-in-time compilation, and runtime safety checks. These layers add overhead, particularly at startup, but they also provide consistency across platforms. Java threads behave the same way on Windows, macOS, and Linux without requiring platform-specific code. This portability is one of Java’s defining strengths, even though it comes with a modest performance cost.

Both versions of the program rely on atomic flags to coordinate thread execution. In C++, the program uses “std::atomic<bool>,” while the Java version uses AtomicBoolean. Both of these constructs compile down to atomic CPU instructions that guarantee visibility and prevent race conditions. However,

the two languages differ in how they handle memory ordering. C++ allows developers to choose from several memory-ordering modes, including relaxed, acquire, release, and sequentially consistent. This flexibility can improve performance when used correctly, but it also increases the risk of subtle concurrency bugs if the wrong ordering is selected. Java simplifies this aspect of concurrency by enforcing sequential consistency for all atomic operations. This means that Java developers do not need to worry about selecting the correct memory-ordering mode, which reduces the likelihood of errors. Ballman (2016) notes that C++ provides powerful concurrency tools but requires careful discipline to avoid mistakes, while Java's more restrictive model trades some performance potential for greater safety and predictability.

The waiting behavior in each program also reflects differences in language design. The C++ version uses "std::this_thread::yield()," which signals to the scheduler that the thread is willing to pause but does not guarantee an actual delay. This approach keeps the thread active and responsive but can consume unnecessary CPU cycles if the waiting period is long. The Java version uses Thread.sleep(10), which explicitly pauses the thread for at least ten milliseconds. This reduces CPU usage during the wait but introduces a slight delay in detecting when the first thread has finished. Both strategies work for a small demonstration program, but in larger systems, both languages offer more efficient synchronization tools. C++ provides condition variables that block a thread until a specific event occurs, while Java offers mechanisms such as CountDownLatch and the wait and notify methods. These tools allow threads to wait efficiently without consuming CPU resources. The differences in waiting strategies illustrate how C++ emphasizes control and performance, while Java emphasizes safety and ease of use.

Memory management is the area where the two languages diverge most dramatically. C++ requires manual memory management, meaning the programmer must allocate and free memory explicitly. Mistakes such as memory leaks, double frees, and use-after-free errors are common sources of security vulnerabilities in C++ applications. Ballman (2016) explains that these issues are among the

most frequent causes of real-world software vulnerabilities and that secure C++ development requires strict adherence to coding standards. Java avoids these problems entirely through automatic garbage collection. The JVM frees memory when objects are no longer in use, eliminating entire categories of memory-related bugs. Oracle's secure coding guidelines emphasize that Java's memory model is intentionally designed to prevent memory corruption vulnerabilities by removing direct pointer manipulation and enforcing strict runtime checks (Oracle, 2022). Although garbage collection can introduce small pauses, these pauses are generally acceptable for most applications. For real-time systems where predictable timing is essential, C++'s deterministic memory behavior may be preferable. However, for general application development, Java's safety benefits outweigh the performance cost.

Type safety and integer behavior also differ significantly between the two languages. Java enforces strict type checking and uses a bytecode verifier to ensure that code cannot perform unsafe casts or invalid memory access. This prevents entire classes of vulnerabilities related to type confusion and memory corruption. C++ allows pointer arithmetic and reinterprets casts, which provide flexibility but can break type safety if misused. Java also defines the behavior of signed integer overflow as wrapping around, while C++ treats signed overflow as undefined behavior. This means that the C++ compiler may optimize away overflow checks entirely, which can lead to unpredictable behavior. Although integer overflow is not a concern in this small program, it is a major difference in larger systems where arithmetic errors can lead to security vulnerabilities. Eck (2015) notes that Java's strict type system and runtime checks are designed to prevent unsafe operations, making Java programs more robust and secure by default.

String handling further highlights the contrast between the two languages. Java strings are immutable and bounds-checked, preventing buffer overflows and making them inherently safer. C++'s "std::string" is safer than raw character arrays but remains mutable and converting to a C-style string reintroduces overflow risks. Many legacy C++ functions such as strcpy and sprintf have historically caused

severe security vulnerabilities, as documented in secure coding standards. Java also avoids format-string vulnerabilities entirely because its printing methods do not interpret format specifiers unless explicitly instructed. One drawback of Java's immutability is that sensitive data cannot be wiped from memory immediately, which is why Oracle recommends using character arrays for passwords and other sensitive information (Oracle, 2022). Still, for general application development, Java's string model is significantly safer than C++'s.

When comparing overall security, the Java implementation is clearly less vulnerable to threats. Java's runtime checks, garbage collection, strict type system, and immutable strings eliminate many vulnerabilities that still exist in C++. C++ can be secure when developers follow strict standards such as the SEI CERT C++ guidelines, but Java's design philosophy prioritizes safety and portability, making it the more secure choice for this project. Ballman (2016) emphasizes that C++ provides powerful tools but requires careful discipline to avoid dangerous pitfalls, while Java's managed environment prevents many of these issues automatically. Eck (2015) similarly highlights Java's emphasis on safety and reliability, which makes it well-suited for applications where security is a priority.

In conclusion, both the Java and C++ versions of the program work correctly and demonstrate the core concurrency concepts required by the assignment. However, the two languages differ significantly in how they handle threading, memory, type safety, and string operations. Java provides strong built-in safety guarantees, while C++ provides maximum control and performance at the cost of increased risk. For most modern applications, Java's safety features make it the better choice, especially when security is more important than raw performance. C++ remains essential for systems where low-level control and maximum speed are required, but it demands far more discipline to use safely. The comparison between the two implementations highlights the trade-offs between performance and security and demonstrates why Java is generally considered the safer language for concurrent programming.

References

Ballman, A. (2016). *SEI CERT C++ coding standard: Rules for developing safe, reliable, and secure systems in C++*. Software Engineering Institute (Carnegie Mellon University). Hanscom, MA.

Eck, D. J. (2015). *Introduction to programming using Java* (7th ed.). CC BY-NC-SA 3.0. http://math.hws.edu/javanotes/

Oracle. (2022, Oct.). *Secure coding guidelines for Java SE*. https://www.oracle.com/java/technologies/javase/seccodeguide.html